

IV Concurso de Modelización Matemática del IMI

Abdelaziz el Kadi Lachehab, Alvaro Barragán Codesal y Alfonso Roldán Otero

1. Introducción

El problema puede abordarse como un grafo con $n + 1$ vértices, donde n es el número de casas, al que se le añade el punto de anclaje. Podemos etiquetar los vértices por el par de coordenadas que representa la posición de las casas y del punto de anclaje en el plano. Los pesos de las aristas que unen los vértices equivalen a la distancia euclídea en $\mathbb{R}^2$ entre los puntos. Puesto que en principio se podría unir directamente mediante un cable cualesquiera vértices, el grafo es completo. Es fácil ver que el camino mínimo no puede contener ciclos, o se estarían añadiendo aristas innecesarias. El camino mínimo será entonces un árbol en el grafo. Un árbol se define como un grafo conexo que no incluye ciclos. Dado un árbol de n vértices, este debe tener $n - 1$ aristas. Entre dos puntos de un árbol existe un único camino que los conecta. Si se añade una arista a un árbol, se forma un ciclo. Un árbol generador de un grafo G es un subgrafo de G tal que es árbol y contiene todos los vértices de G . El árbol generador mínimo de G es un árbol generador de G tal que la suma de los pesos de sus aristas es mínima. El siguiente teorema nos dice además que siempre existe un árbol mínimo generador:

–**Teorema 1:** En todo grafo ponderado existe algún árbol generador mínimo.

Demostración: Dado un grafo con un conjunto finito de vértices, el cardinal del conjunto de subgrafos es también un número finito. Por tanto, existe un valor mínimo para la suma de los pesos de las aristas.

Vamos a introducir a continuación un algoritmo que será clave para los sucesivos apartados: el algoritmo de Prim. Mediante el algoritmo de Prim se puede encontrar el árbol generador mínimo de un grafo, que sabemos que existe por el teorema anterior. Enunciamos el algoritmo para un grafo G conexo, finito con n vértices etiquetados y una función de peso w :

–Paso 1: Se elige una arista con peso mínimo. Sea a dicha arista. Se define $T_1 = a$ como el conjunto de aristas del árbol.

–Paso $k < n$: Se supone construido en el paso $(k - 1)$ el árbol T_{k-1} . Para construir el árbol se elige una arista de peso mínimo incidente con algún vértice de T_{k-1} de manera que no se forme un ciclo. Denotamos por k a esa arista. Se actualiza el árbol añadiendo la arista k , de modo de que $T_k = T_{k-1} \cup k$.

–El algoritmo finaliza tras $n - 1$ pasos.

El siguiente teorema nos permitirá emplear el algoritmo de Prim para la resolución del problema:

–**Teorema 2:** El árbol T obtenido mediante el algoritmo de Prim es un árbol generador mínimo, es decir, para cualquier otro árbol generador B del grafo G se cumple:

$$w(T) \leq w(B)$$

Donde $w(T)$ y $w(B)$ denotan la suma de los pesos de las aristas de los árboles T y B respectivamente.

2. Planteamiento general

2.1. ¿Por qué tienen que ser rectas?

Supongamos que en el camino mínimo existe una curva γ_1 en el plano distinta de un segmento de $\mathbb{R}^2$ tal que dos puntos del grafo están unidos mediante dicha curva. Entonces, por ser el camino del grafo el camino de mínima longitud, en particular la longitud de la curva será menor que la de otra curva γ_2 entre esos mismos dos puntos, en particular que el segmento que los une. Pero esto es falso, ya que la distancia euclídea entre dos puntos viene dada por el vector que los une. Luego todas las curvas que unen vértices del grafo deben ser segmentos de recta.

El problema de la mínima distancia entre dos puntos puede abordarse también como un problema elemental de cálculo de variaciones, véase [1].

2.2. ¿Por qué no se van a cruzar?

La demostración se basa en el Teorema 2 y en nociones geométricas básicas de los cuadriláteros. Sabemos del apartado anterior que todas las aristas son segmentos. Por lo tanto que dos cables se crucen es equivalente a que dos segmentos intersequen. Sean $\{A, B, C, D\}$ cuatro puntos del grafo de manera que el vértice A fue anido con el vértice B en el paso $Y \in \mathbb{N}$ del algoritmo de Prim y el punto C , unido a los puntos A y B en un paso del algoritmo no necesariamente mediante una arista, se une con el punto D en el paso $X > Y$. Los cuatro puntos forman un cuadrilátero convexo. Veamos que esto no es posible. Se distinguen 3 casos:

i) $d(D, C) < d(D, A)$. Por un argumento geométrico se pueden dirimir los dos casos que se pueden presentar: $d(D, A) \leq d(A, B)$, y entonces el algoritmo se aplicó mal en el paso Y , ya que el punto D estaba a menor distancia del punto A que el punto B ; o bien $d(D, A) > d(A, B) \Rightarrow d(D, C) > d(D, B)$. Supuesto que el algoritmo se ejecutó correctamente en el paso Y , se está ejecutando mal el paso X , ya que el punto B está a menor distancia de D que el punto C .

ii) $d(D, C) > d(D, A)$. Entonces el algoritmo se está aplicando mal en el paso X . El punto D estaba a menor distancia del punto A que del punto C .

iii) $d(D, C) = d(D, A)$. En este caso, la única forma de que se corten ambos segmentos es que el punto D se encuentre sobre el segmento que une A y B . Pero entonces el punto D ya está unido al árbol y no se debía conectar con C o se genera un ciclo.

Discutidos todos los casos, se concluye que dos cables no pueden cortarse. Aplicando la demostración sobre segmentos dos a dos, se demuestra también que en general ningún número de cables puede cortarse o la longitud del árbol no sería mínima.

3. Modelado y solución del problema

3.1. Propiedades de la red

Debido a las propiedades del que debe cumplir la red, el punto de enganche no juega un papel especial en el grafo, es decir, el árbol generador mínimo que se obtiene permutando el punto de enganche con una casa coinciden.

El problema que se presenta es conocido como problema del árbol generador mínimo (*Minimum Spanning Tree Problem*), que consiste en unir los nodos de un grafo ponderado y no direccionado mediante un camino, de manera que la suma de las pesos (las longitudes en este caso) de las aristas sea mínima. Como se indicó en la introducción, el camino mínimo que atraviese todos los vértices del grafo debe ser un árbol. Este árbol verificará las propiedades ya expuestas, además de, por lo demostrado en 2, ser todas sus aristas rectas y no cruzarse entre ellas.

3.2. Modelo matemático

Un grafo se define como un par (V, A) , donde V es el conjunto de vértices del grafo (las casas y el punto de enganche en nuestro caso) y A es el conjunto de aristas del grafo, donde una arista $a \in A$ es un par no ordenado de vertices de V , en nuestro caso se tratan de los cables que conectan las casas.

Como hemos indicado antes, el grafo que buscamos es un árbol $A = (C, R)$ donde C es el conjunto de casas y el enganche, y R es el conjunto de aristas que forman la red. Observemos que $|R| = |C| - 1$. El algoritmo de Prim es probablemente el método más fácil de computar (y por tanto más rapido) para encontrar una solución al problema, sin embargo, podemos modelizar este problema como un problema de programación lineal, para ello haremos uso de la siguiente caracterización para conocer las restricciones del problema:

$$G = (V, A) \text{ es un árbol} \Leftrightarrow |A| = |V| - 1 \text{ y no hay ciclos en } G$$

Además sabemos que G no tiene ciclos si $\forall U \subset V, |B| \leq |U| - 1$ donde $B \subset A$ es el conjunto de aristas del árbol tales que ambos puntos están en U .

Tenemos así el siguiente problema de programación lineal:

Dado un conjunto de vértices C , buscamos el conjunto R de aristas que unen algunos vértices de forma que el árbol $A = (C, R)$ tenga distancia mínima. Definimos $a_{i,j}$ como la variable de activación de la arista que une los puntos $i, j \in C$ y $d_{i,j}$ como el valor real de la distancia entre los puntos $i, j \in C$. ($a_{i,j} = a_{j,i}$ y $d_{i,j} = d_{j,i}$). Donde $a_{i,j} \in \{0, 1\}$, $\forall i, j \in C$ y $d_{i,j} \in \mathbb{R}$, $\forall i, j \in C$:

$$\min z = \sum_{i,j \in A} a_{ij} d_{ij}$$

Sujeto a:

$$\sum_{i,j \in A} a_{ij} = |C| - 1$$

$$\sum_{i,j \in U} a_{ij} \leq |U| - 1 \quad \forall U \subset C$$

El conjunto S de los valores $a_{i,j} \in \{0,1\}$ que minimizan la función de distancia total z indicará cuales son las redes que debemos construir, es decir, si el valor de $a_{i,j} = 1$ en S indicará que la red entre la casa i y la casa j debe estar conectada.

Como hemos mencionado anteriormente, existe un algoritmo en redes apropiado para resolver este problema: el Algoritmo de Prim, explicado en la sección 1.1.

3.3. Método para calcular la red de distancia mínima

Como hemos indicado anteriormente, haremos uso del algoritmo de Prim, programado en Python 3.7 como todo el código que adjuntaremos. Dado un conjunto C , la función $prim(C)$ devuelve el árbol (nodos y aristas), junto con la longitud de este.

```
"""
ALGORITMO DE PRIM

"""
def prim(c: list):
    #Realiza el algoritmo de Prim dado un conjunto de puntos de R^2
    #Devuelve el arbol de minima distancia y la longitud de red
    p = [] + c #Copia del conjunto de puntos
    nodos = [] #Nodos del arbol
    aristas = [] #Aristas del arbol

    nodos.append(p.pop(0))
    t = minima_distancia(nodos, p)
    nodos.append(p.pop(p.index(t[1])))
    aristas.append([nodos[0], nodos[1]])

    while len(p) != 0:
        t = minima_distancia(nodos, p)
        nodos.append(p.pop(p.index(t[1])))
        aristas.append([t[0], t[1]])

    return [nodos, aristas, longitud(aristas)]
```

4. Mejorando la red óptima

4.1. Ejemplo de un punto de Steiner

Veamos como reducir la longitud del árbol generador mínimo mediante un punto de Steiner. Sea un triángulo equilátero de lado a . Entonces la longitud del árbol $L(N) = 2a$. Añadiendo un punto en el ortocentro del triángulo al árbol, la nueva longitud del árbol generador

mínimo es $L(N') = 3a\frac{\sqrt{3}}{3} = \sqrt{3}$. Luego el cociente entre las longitudes es $\frac{L(N')}{L(N)} = \frac{\sqrt{3}}{2}$. De hecho, no existe un árbol tal que el ratio de las longitudes del árbol generador mínimo original y el árbol generador mínimo con puntos de Steiner pueda reducirse más que en el triángulo equilátero (véase la siguiente cuestión).

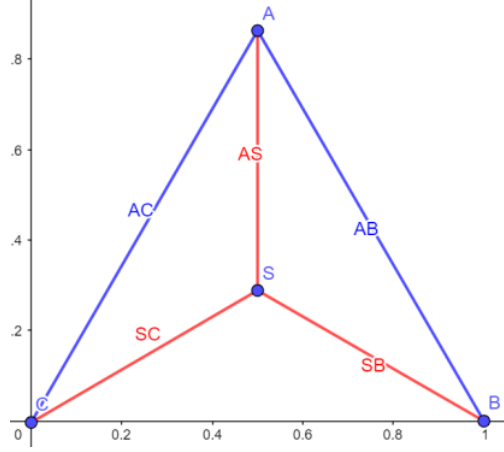


Figura 1: Grafo de 3 puntos en disposición de triángulo equilátero. En azul el mínimo camino del árbol, en rojo el camino tras añadir el punto de Steiner.

4.2. Cota del ratio de Steiner

Un concepto importante a la hora de valorar la eficacia de la solución encontrada es el ratio de Steiner. En el problema euclídeo, se define como el cociente de la longitud del árbol de Steiner y la longitud del árbol de camino mínimo. Tener una cota inferior del ratio de Steiner es útil ya que permite valorar cuánto se ha acercado la solución encontrada a esa cota. Gilbert y Pollak conjeturaron en 1964 que nunca se podría encontrar un ratio de Steiner inferior a $\frac{\sqrt{3}}{2}$. Sucesivas cotas inferiores fueron demostrándose, hasta que en 1992, Hwang[3] demostró que el ínfimo del ratio de Steiner es exactamente $\frac{\sqrt{3}}{2}$. Para un triángulo equilátero, en particular, se verifica como se vio en la cuestión anterior que el ratio de Steiner es igual a $\frac{\sqrt{3}}{2}$. Demostremos la desigualdad para triángulos en general:

—**Teorema** : Para un árbol de mínima distancia con tres vértices, el árbol mínimo de Steiner verifica: $L(N') \leq \frac{\sqrt{3}}{2}L(N)$, donde $L(N)$ es la longitud del árbol inicial y $L(N')$ es la longitud del árbol mínimo de Steiner.

Demostración: Sean $\{a, b, c\}$ puntos terminales con un árbol mínimo de Steiner dado por $S = [a, s] \cup [b, s] \cup [c, s]$. Sin pérdida de generalidad, supongamos que la longitud de la arista $[a, s]$ no es mayor que la de las otras dos aristas. Si $s = a$, entonces $S = T$, donde T es el árbol de camino mínimo generado por $\{a, b, c\}$ y el teorema es cierto. Suponamos por tanto que $s \neq a$. Sea $b'(c')$ un punto en la arista $[b, s]([c, s])$ tal que $|b', s| = |a, s|(|c', s| = |a, s|)$. Entonces, puesto que $\{a, b', c'\}$ forman un triángulo equilátero, se verifica que:

$$\begin{aligned} |S| &= |b, b'| + |c, c'| + |a, s| + |b', s| + |c', s| \\ &= |b, b'| + |c, c'| + \frac{\sqrt{3}}{2}(|a, b'| + |a, c'|) \geq \frac{\sqrt{3}}{2}(|a, b'| + |a, c'|) \geq \frac{\sqrt{3}}{2}|T| \end{aligned}$$

Y queda demostrado el teorema.

4.3. El cálculo de N' es un problema NP-duro

El problema de encontrar un MST (Minimal Steiner Tree) se trata de un problema NP-Duro si dado un algoritmo A que resuelva nuestro problema, podemos reducir un problema NP-Completo a nuestro problema y aplicar ese algoritmo para su resolución en un tiempo polinómico. Este problema NP-Completo existe, se trata del problema *Exact Cover by 3-Sets* y en [2] explica su reducción.

4.4. Propiedades del punto añadido

Vamos a utilizar el punto de Fermat. Este punto se define como el punto de un triángulo tal que la suma de la distancia entre este punto y los vértices del triángulo es mínima. El punto añadido para el triángulo equilátero era precisamente el punto de Fermat para dicho triángulo. Dicho problema es equivalente a encontrar el camino de mínima longitud de Steiner para un grafo con tres puntos. El punto de Fermat verifica además:

i) Si todos los ángulos del triángulo son menores que 120° , entonces el ángulo subtendido por el punto de Fermat y cada uno de los vértices forma 120° .

ii) Si alguno de los ángulos del triángulo es mayor o igual a 120° , el punto de Fermat coincide con el vértice correspondiente a dicho ángulo. El objetivo es aproximar cuanto sea posible los ángulos que forman los puntos de Steiner con los terminales a 120° .

Además, el punto de Fermat obviamente verifica, como se pide en el enunciado, que su grado es 3.

4.5. Red de longitud mínima con un punto de Fermat

Para calcular el camino mínimo bajo las condiciones de 4.4., vamos a emplear el punto de Fermat recursivamente. Dada la matriz con los n puntos representando las posiciones en el plano de las casas y el punto de anclaje, para cada terna de puntos no ordenados se calcula el punto de Fermat. Hallado un punto de Fermat para una terna, se añade dicho punto a la matriz de puntos y se ejecuta el algoritmo de Prim sobre los $n + 1$ puntos, calculando la longitud mínima para el nuevo grafo. Calculados los $\binom{n}{3}$ árboles generadores mínimos asociados a cada punto de Fermat, se escoge aquel cuya longitud sea menor, siendo esta la solución exacta al problema.

La función *steiner_points* devuelve el conjunto *sp*, que son todos los puntos de Fermat posibles a añadir, dicha función elimina también todos los puntos repetidos. La función *steiner_best* calcula todos los puntos de Fermat y escoge aquel que genera el árbol cuya longitud es menor.

```
"""
UN PUNTO DE STEINER
"""

def steiner_point(p: list):
    #Dado un conjunto de puntos calcula todos los puntos de Fermat
    #Recorre todas las ternas y almacena el potencial punto de Steiner
```

```

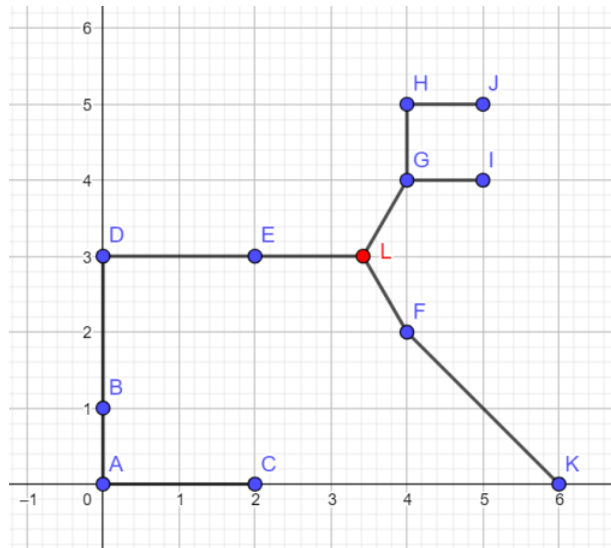
#Obs: obtendremos len(p) sobre 3 puntos de Fermat
c = [] + p
sp = []

for i in c:
    c2 = c[c.index(i)+1:]
    for j in c2:
        if len(c2)>1:
            c3 = c2[c2.index(j)+1:]
            for k in c3:
                sp.append(fermat(i,j,k))

return sp

def steiner_best(p: list):
    #De todos los potenciales puntos de Steiner encuentra el de menor
    longitud
    #Devuelve el arbol tras a adir ese punto junto con su longitud y
    eficiencia
    sp = steiner_point(p)
    l = prim(p)
    m = ["Original", l]
    for i in sp:
        t = prim(p + [i])
        if m[1][1] > t[1]:
            m = [i,t]
    return [m, m[1][1]/l[1]]

```



4.6. Algoritmos de aproximación a N'

El problema al eliminar la restricción de 4.6 es un problema de Steiner, que consiste en encontrar el árbol mínimo de Steiner (*Steiner Minimal Tree*). El conjunto de vértices G se divide en dos subconjuntos: los puntos terminales T dados por los que debe pasar el árbol de Steiner, y los puntos de Steiner S por los que puede pasar el árbol si haciéndolo se reduce la suma total de la función peso. Existen dos versiones del problema: el problema de los árboles mínimos de Steiner, donde el conjunto G está dado y se debe encontrar el árbol de Steiner que

minimice la suma de los pesos de las aristas del árbol, y el problema euclídeo de los árboles mínimos de Steiner, donde el subconjunto T está dado, pero S podría ser en principio todo el plano y los pesos vienen dados por la norma euclídea. Este segundo caso es nuestro problema. Para disminuir la longitud árbol generador mínimo, se pueden añadir a lo sumo $n-2$ puntos de Steiner. Además, un árbol euclídeo mínimo de Steiner verifica:

i) El árbol euclídeo mínimo de Steiner tiene grado máximo tres.

ii) *Propiedad de grado*: Los puntos de Steiner siempre tienen grado tres. Un punto de Steiner de grado uno puede ser eliminado junto con su arista para reducir la longitud del árbol. Un punto de Steiner de grado dos podría ser eliminado junto con sus aristas por una única arista uniendo los dos puntos sin incrementar la longitud del árbol.

iii) *Propiedad de ángulo*: El ángulo que forman dos aristas cualesquiera que parten del mismo punto es mayor o igual a 120° , y exactamente 120° en los puntos de Steiner. Para incorporar esta propiedad en un algoritmo, se debería permitir que el ángulo (medido en grados) se encuentre en un cierto intervalo $[120 - \epsilon, 120 + \epsilon]$.

Para resolver el problema vamos a plantear diversos algoritmos, ordenados siguiendo el proceso de refinamiento que atravesaron. Todos ellos devuelven un vector de tres elementos, donde el primero es el conjunto de nodos (puntos de $\mathbb{R}^2$), el segundo es el conjunto de aristas (pares de nodos) y el tercer elemento del vector es la longitud total del árbol.

–**Algoritmo 1**: Está basado en el algoritmo presentado en 4.5. Introducimos el concepto de *tolerancia*, este valor $ep > 0$ indicará cual será la mejora mínima en cada iteración con el fin de evitar bucles que el programa entre en bucle. El algoritmo calculará el mejor punto de Fermat entre los n puntos de la matriz inicial que incluye las casas y el punto de anclaje, y lo incorpora a la matriz de puntos. Se vuelve a calcular el mejor punto de Fermat entre $n + 1$ puntos. A continuación se calcula el ratio de Steiner entre la nueva longitud mínima y la longitud mínima del paso anterior. Si el cociente es inferior a $1 - ep$, entonces se añade el punto de Fermat correspondiente al árbol de longitud mínima y se incorpora a la matriz de puntos.

Este algoritmo tiene un coste computacional muy alto (y de crecimiento exponencial) y puede entrar en bucles si la tolerancia es muy baja, sin embargo, con él hemos conseguido la distancia mínima en un conjunto C de 34 nodos $longitud(steiner_all(C)) = 111,22$. (Anexo)

El algoritmo tiene una posible mejora en la búsqueda de el mejor punto de Fermat añadir, si almacenamos la informacion de los puntos calculados no sería necesario calcularlos en cada iteración.

```
"""
Algoritmo de generacion por puntos unicos de Steiner (usando steiner_best)
"""

def steiner_all(p: list):
    #Dado un conjunto de R^2 calcula el punto de Steiner a aadir, lo
    #ade
    #al conjunto original y vuelve a calcular otro punto de Steiner (m
```



```

    veces)
    ep = 0.005 #Tolerancia en la eficiencia
    b = True #Variable de salida del bucle
    n = 0 #Iteraciones
    m = 11 #Maximo de iteraciones, no debe alcanzar len(p)-2 (Teorema)
    c = [] + p #Copia de puntos
    while b and n < m:
        st = [] + steiner_best(c)
        if st[1] >= (1-ep): #Break por eficiencia
            b = False
        else:
            c = c + [st[0][0]]
            n = n + 1
    return prim(c)

```

—**Algoritmo 2** : El siguiente algoritmo realiza una construcción del árbol de manera mixta, siendo un algoritmo híbrido entre la construcción de puntos de Fermat y el algoritmo de Prim. En cada iteración añadimos se verifica si es mejor (localmente) añadir al árbol el punto que proporciona el algoritmo de Prim uniéndolo mediante la arista de menor longitud o mediante un punto de Fermat entre los extremos de dicha arista y un punto adyacente del árbol al extremo de la arista que ya está incluido en la matriz de puntos. Para ello evaluamos la expresión

$$d(t_1, t_0) + d(t_0, f) > d(t_1, f) + d(t_0, f) + d(r, f)$$

Donde $d(x, y)$ es la norma euclídea, t_0, r son puntos ya pertenecientes al árbol, t_1 es el punto a añadir por el algoritmo de Prim a t_0 y f es el punto de Fermat de los tres anteriores. Si se cumple la desigualdad, entonces localmente es mejor añadir el punto de Fermat al árbol para añadir t_0 .

No obstante, puede suceder que el algoritmo sea óptimo localmente pero no globalmente, de manera que aunque añadir el punto de Fermat sea ventajoso en un paso, la longitud total no disminuya. Además, al añadir nuevos puntos de Fermat, los puntos de Fermat que se añadieron previamente pueden perder la propiedad de grado y de ángulo de los puntos de Steiner. Una posible mejora al algoritmo sería, por tanto, comprobar que todos los puntos en el conjunto de puntos de Fermat deban cumplir que todos sus ángulos sean de 120° y su grado sea 3. Si algún punto no lo cumple, se recalcula el árbol hasta que todos los puntos del conjunto cumplan ambas propiedades.

Dado un conjunto de casas C y un elemento donde iniciar el algoritmo $i \in C$, tenemos que la función definida $mix_st(C, i)$ genera la aproximación al MST de C empezando por el nodo i , una mejora implementada de este algoritmo es otra función que prueba iniciando $mix_st(C, i)$, $\forall i \in C$, devolviendo el mejor árbol de los calculados en esa iteración, se trata de la función $mix_st_iter(C)$.

```

"""
Algoritmo de busqueda mixta con criterio local
"""
#Algoritmo empezando por el i-esimo nodo
def mix_st(c: list, i):
    #Dado un conjunto de puntos de R^2 calcula un MST aproximado por
    seleccion

```

```

#Devuelve la aproximacion al MST con su longitud
p = [] + c #Copia del conjunto de puntos
aristas = [] #Aristas del arbol
nodos = [] #Nodos del arbol
sp = [] #Puntos de Steiner

nodos.append(p.pop(i)) #Iniciamos el algoritmo en el i-esimo nodo
t = minima_distancia(nodos, p)
nodos.append(p.pop(p.index(t[1])))
aristas.append([nodos[0],nodos[1]])

while len(p) != 0:
    t = minima_distancia(nodos,p)
    r = ady(t[0],aristas)
    s = fermat(t[0], t[1], r)
    #Elegimos si localmente es mejor agregar un punto de Fermat o el
    siguiente nodo
    if dist(t[1],t[0]) + dist(t[0],r) > dist(t[1],s) + dist(t[0],s) +
    dist(r,s):
        if [t[0],r] in aristas:
            aristas.remove([t[0],r])
        else:
            aristas.remove([r,t[0]])
            aristas.append([t[0],s])
            aristas.append([t[1],s])
            aristas.append([r,s])
            sp.append(s)
            nodos.append(s)
            nodos.append(p.pop(p.index(t[1])))
    else:
        nodos.append(p.pop(p.index(t[1])))
        aristas.append([t[0],t[1]])

    return [nodos, aristas, longitud(aristas)]

#Algoritmo de iteracion que prueba con todos los posibles inicios de mix_st
def mix_st_iter(c):
    #Dado un conjunto de puntos de R^2 calcula un MST aproximado iterando
    todos
    #los posibles nodos de inicio
    best = mix_st(c,0)
    for i in range(1,len(c)):
        t = mix_st(c,i)
        if best[2] > t[2]:
            best = t
    return best

```

—**Algoritmo 3** : Debido a la complejidad de comprobar en cada iteración si los puntos de Steiner conservaban sus propiedades de ángulo y grado, decidimos intentar mejorar los resultados mediante un tercer algoritmo que además da pie a la entrada del Machine Learning en este problema. El algoritmo funciona de la misma manera que el Algoritmo 2, solo que a diferencia que en vez de usar un criterio local para decidir si usar el algoritmo de Prim o añadir el punto de Fermat calculado (desigualdad de distancias dadas en la explicación del Algoritmo 2) se va usar un criterio de *confianza*. La confianza va a ser un valor comprendido entre 0 y 1 que determine el porcentaje de veces que vamos a elegir el punto de Fermat sobre

seguir el camino dado por el algoritmo de Prim. Esto generará un camino casi aleatorio.^{en} el árbol, de manera que si iteramos it veces podemos encontrar un camino que sea el mejor de manera global.

Dado un conjunto de casas C y un elemento donde iniciar el algoritmo $i \in C$, las ventajas de este algoritmo sobre el anterior es que, por la forma en la que hemos definido la función, va a obtener una aproximación al MST mejor o igual que $mix_st(C, i)$. Definimos así la función $random_mix_st(C, i)$ y su mejora (recorriendo todas los posibles nodos i) $random_mix_st_iter(C)$.

```
"""
Algoritmo de busqueda mixta con seleccion por confianza (MONTECARLO)
"""

#Algoritmo de busqueda mixta empezando por el k-esimo nodo

def random_mix_st(c: list, k):
    #Dado un conjunto de puntos de R^2 calcula un MST aproximado por
    #seleccion
    #Devuelve el MST y longitud
    p = [] + c #Copia del conjunto de puntos
    aristas = [] #Aristas del arbol
    nodos = [] #Nodos del arbol
    sp = [] #Puntos de Steiner
    #Valor entre 0 y 1 que determina la confianza en los puntos de Fermat
    confianza = 0.7
    #Numero de arboles pseudoaleatorios creados
    it = 10000
    nodos.append(p.pop(k))
    t = minima_distancia(nodos, p)
    nodos.append(p.pop(p.index(t[1])))
    aristas.append([nodos[0], nodos[1]])
    ini = mix_st(c, k)
    #Iniciando el output de esta manera nos aseguramos que esta funcion
    #mejore
    #siempre el resultado de mix_st
    best = [ini[0], ini[1], ini[2]]
    for i in range(it):
        while len(p) != 0:
            t = minima_distancia(nodos, p)
            r = ady(t[0], aristas)
            s = ffermat(t[0], t[1], r)
            rand = random.random()
            #Eleccion por confianza de elegir un punto de Fermat
            #Evitamos dobles aristas
            if rand < confianza and not((s in nodos) or (s in p)):
                if [t[0], r] in aristas:
                    aristas.remove([t[0], r])
                else:
                    aristas.remove([r, t[0]])
                    aristas.append([t[0], s])
                    aristas.append([t[1], s])
                    aristas.append([r, s])
                    sp.append(s)
                    nodos.append(s)
                    nodos.append(p.pop(p.index(t[1])))
```

```

        else:
            nodos.append(p.pop(p.index(t[1])))
            aristas.append([t[0],t[1]])
    if best[2]>longitud(aristas):
        best=[nodos, aristas, longitud(aristas)]

    return best

#Algoritmo de iteracion que prueba con todos los inicios de random_mix_st
def random_mix_st_iter(c):
    #Dado un conjunto de puntos de R^2 calcula un MST aproximado iterando
    todos
    #los posibles nodos de inicio en random_mix_st(c,k)
    best = random_mix_st(c,0)
    for i in range(1,len(c)):
        t = random_mix_st(c,i)
        if best[2] > t[2]:
            best = t
    return best

```

4.7. Aplicación del algoritmo para el conjunto de casas C

Aplicamos los algoritmos al conjunto de C , formado por 10 casas y un punto de anclaje:

$$C = [[0, 0], [0, 1], [2, 0], [0, 3], [2, 3], [4, 2], [4, 4], [4, 5], [5, 4], [5, 5], [6, 0]]$$

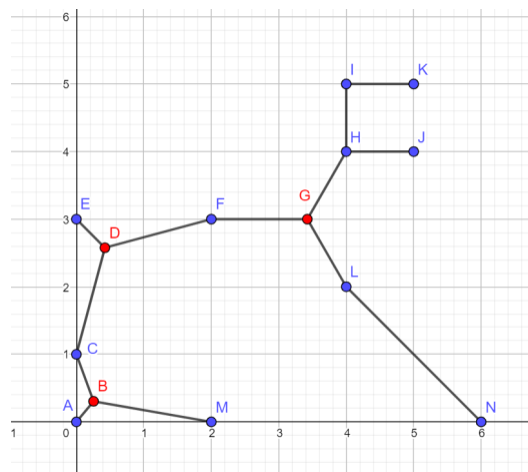
A continuación se adjuntan los resultados:

-Algoritmo de Prim:

$$Longitud = 17,06$$

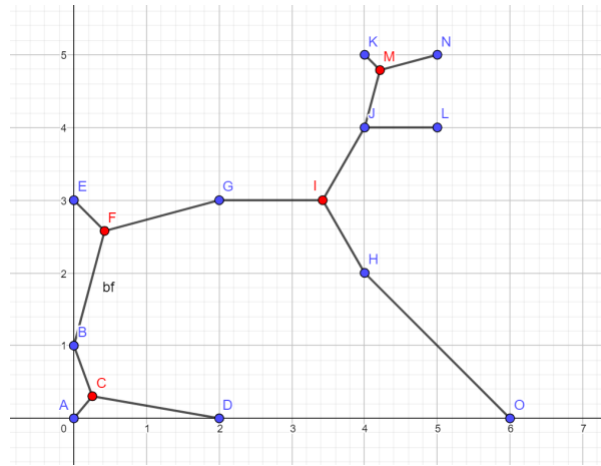
-Algoritmo 1:

$$Longitud = 16,33$$



-Algoritmo 3:

$$Longitud = 16,27$$



En este caso, el algoritmo 3 no es capaz de mejorar el resultado proporcionado por el algoritmo 2. Si aumentamos el número de casas, se observa como el algoritmo 3 es capaz de mejorar los resultados del algoritmo 2.

5. Anexo

5.1. Funciones auxiliares

Para la limpieza del código, hemos hecho uso de una funciones auxiliares previamente definidas:

```
"""
FUNCIONES AUXILIARES
"""
from math import*
import numpy

def dist(x,y):
    #Norma euclidea en R^2
    return sqrt((x[0] - y[0])**2 + (x[1] - y[1])**2)

def alineados(a,b,c):
    #Comprueba si 3 puntos de R^2 se encuentran alineados
    ep = 0.00000001 #Tolerancia
    if a==b and b==c:
        return True
    else:
        ab=[b[0]-a[0],b[1]-a[1]]
        cb=[b[0]-c[0],b[1]-c[1]]
        r = (numpy.dot(ab,cb))/(dist(a,b)*dist(b,c))
        return abs(r)<1+ep and abs(r)>1-ep #Devuelve False o True

def fermat(z,a,b):
    #Calcula el punto de Fermat dados 3 puntos
    #En caso de estar los 3 puntos alineados devuelve el punto medio
    if alineados(z,a,b):
        mid=[]
        mid.append([z[0],a[0],b[0]])
        mid.append([z[1],a[1],b[1]])
        mid[0].sort()
        mid[1].sort()
        return [mid[0][1],mid[1][1]]
    else:
        c=complex;T=lambda A,B:abs(c*(A)-c*(B));d=T(z,a);e=T(z,b);f=T(a,b);
        g=[d,e,f];h=max(g);g.remove(h);
        i=acos((sum(i*i for i in g)-(h*h))/(2*g[0]*g[-1]));_=[[z,a],[z,b],[
        a,b]];j,s,t=cos,sin,atan;N=[[b,a]for a,b in zip([b,a,z],[max(i,
        key=i.get)if i!=''else''for i in [(g[0]+(h*j(t(1))),g[1]+(h*s(t
        (1)))):T(k,(g[0]+(h*j(t(1))),g[1]+(h*s(t(1))))),(g[0]-(h*j(t(1)
        )),g[1]-(h*s(t(1)))):T(k,(g[0]-(h*j(t(1))),g[1]-(h*s(t(1))))}
        if l else{(g[0]+h,g[1]):T(k,(g[0]+h,g[1])),(g[0]-h,g[1]):T(k,(g
        [0]-h,g[1]))}if l==0else''for g,h,l,k in zip([(a[0]+b[0])/2,(a
        [1]+b[1])/2)for a,b in _],[3**0.5)*(i/2)for i in [d,e,f],[-1/p
        if p else''if p==0else 0for p in [(a[1]-b[1])/(a[0]-b[0])]if a
        [0]-b[0]else''for a,b in _],[b,a,z]]))if b!='';I=N[0][0][1];
```

```

        J=N[0][0][0];K=N[1][0][1];G=N[1][0][0];A=(N[0][1][1]-I)/(N
        [0][1][0]-J);B=I-(A*J);C=(K-N[1][1][1])/(G-N[1][1][0]);D=K-(C*G
        );X=(D-B)/(A-C);Y=(A*X)+B
    return [[X,Y],[[a,b][h==d],z][h==f]][i>2.0943]

def minima_distancia(a,b):
    #Devuelve la minima distancia entre un punto de a y b
    #a y b deben ser conjuntos de puntos de R^2
    m = [a[0],b[0],dist(a[0],b[0])]
    for i in a:
        for j in b:
            d = dist(i,j)
            if m[2] > d:
                m = [i,j,d]

    return m

def ady(p,a):
    #Devuelve un punto m arbitrario que esta conectado a p por una arista
    de a
    m = []
    for i in a:
        if p in i:
            m = i[(1 - i.index(p))]
            break
    return m

def longitud(a):
    #Calcula la longitud total del grafo dado el conjunto de aristas a
    l = 0
    for i in a:
        l = l + dist(i[0],i[1])
    return l

```

Bibliografía

- [1] Taylor, John R. (John Robert), 1939-. (2005). Classical mechanics. Sausalito, Calif. :University Science Books,
- [2] Steiner Tree NP-completeness Proof, Alessandro Santuari
- [3] Frank K. Hwang, Dana S. Richards, Pawel Winter, The Steiner Tree Problem. Annals of discrete mathematics, (1992)